Lecture 18 (B17): Physics-Informed Neural Networks — Foundations

DEQN vs PINN, automatic differentiation for PDEs, training pipeline, architecture

Simon Scheidegger

HEC, University of Lausanne

Outline

1. **From DEQNs to PINNs**
   Bridging discrete and continuous time; the PINN loss.
2. **PINN Fundamentals**
   Automatic differentiation for PDEs, collocation, training loop.
3. **Soft vs. Hard Boundary Conditions**
   Trial solutions, 2D Poisson, practical guidance.
4. **Economic Applications: HJB Equation**
   Cake-eating problem, scaled trial solution + hard BCs, Adam $\rightarrow$ L-BFGS, analytical benchmark; DGM as optional high-D architecture.
5. **Financial Applications: Black–Scholes**
   Option pricing via PINNs, comparison with analytical formula.

Roadmap of This Lecture (75 min)

From DEQNs to PINNs (10 min)

- Discrete vs. continuous time
- The PINN loss function
- DEQN vs. PINN comparison

PINN Fundamentals (20 min)

- Autograd for PDEs
- Collocation and training
- Warm-up: 1D ODE example

Boundary Conditions (15 min)

- Soft vs. hard enforcement
- Trial solutions
- 2D Poisson example

Econ & Finance (30 min)

- HJB / cake-eating: trial solution + hard BCs
- Adam warm-start, L-BFGS polish, FP64
- Black–Scholes PDE; DGM as optional architecture

Part I

From DEQNs to PINNs

Recall: The DEQN Loss

Yesterday: we trained a neural network $\mathcal{N}_\theta$ to satisfy economic equilibrium conditions in discrete time:

$$\boxed{\ell_\theta^{\text{DEQN}} = \frac{1}{N}\sum_{i=1}^{N}\left\| G\left(\boldsymbol{x}_i, \mathcal{N}_\theta(\boldsymbol{x}_i)\right)\right\|^2}$$

- $G(\cdot) = 0$: Euler equations, budget constraints, market clearing
- Training points $\boldsymbol{x}_i$: simulated from the model's own dynamics
- No labeled data needed—structural equations *are* the loss

Today's question: What if the economic model is formulated in continuous time?

Discrete vs. Continuous Time

Discrete Time (DEQN)

Euler equation:

$$u'(c_t) = \beta\,\mathbb{E}[u'(c_{t+1})\,R_{t+1}]$$

State transition:

$$\boldsymbol{x}_{t+1} = h(\boldsymbol{x}_t, p(\boldsymbol{x}_t), \varepsilon_{t+1})$$

Loss: algebraic residual $G(\cdot) = 0$

Continuous Time (PINN)

HJB equation:

$$\rho\,V = \max_c \{u(c) + V_a\,\dot{a}\}$$

State evolution:

$$\mathrm{d}a = (ra - c)\,\mathrm{d}t + \sigma\,\mathrm{d}W$$

Loss: PDE residual $\mathcal{R}(\cdot) = 0$

Key shift: from algebraic residuals to **differential** residuals—we need derivatives of the neural network.

Why Continuous Time Matters

Many frontier models are formulated in continuous time:

- Heterogeneous-agent models: Achdou, Han, Lasry, Lions & Moll (2022)—income and wealth distribution dynamics via HJB–KFE systems
- Mean-field games: optimal policy with a continuum of interacting agents
- Asset pricing: Black–Scholes, Heston, jump-diffusion models
- Macro-finance: Brunnermeier & Sannikov (2014), He & Krishnamurthy (2013)

Central bank relevance

Continuous-time models are the workhorse for financial stability analysis, option pricing, and monetary transmission in modern central banking.

The PINN Loss Function

Core idea (Raissi, Perdikaris & Karniadakis, 2019):

Given a PDE $\mathcal{D}[u](\boldsymbol{x}) = 0$ on domain Ω with boundary conditions $\mathcal{B}[u](\boldsymbol{x}) = 0$ on $\partial\Omega$, define:

$$\boxed{\ell_\theta^{\mathrm{PINN}} = \underbrace{\frac{1}{N_r}\sum_{i=1}^{N_r}\left|\mathcal{D}[\mathcal{N}_\theta](\boldsymbol{x}_i^r)\right|^2}_{\text{PDE residual}} + \underbrace{\frac{\lambda}{N_b}\sum_{j=1}^{N_b}\left|\mathcal{B}[\mathcal{N}_\theta](\boldsymbol{x}_j^b)\right|^2}_{\text{boundary/initial conditions}}}$$

- $\mathcal{N}_\theta$: neural network with parameters θ
- $\boldsymbol{x}_i^r$: collocation points in the interior of Ω
- $\boldsymbol{x}_j^b$: points on the boundary $\partial\Omega$
- Derivatives $\mathcal{D}[\mathcal{N}_\theta]$ computed via automatic differentiation

Warm-up ODE: for $u''(x) = -1$ on $(0, 1)$ with $u(0) = u(1) = 0$, use $\mathcal{D}[u] = u'' + 1$ in the interior and $\mathcal{B}[u] = (u(0), u(1))$ on the boundary.

PAUSE Finger Exercise: Writing a PINN Loss

Exercise (5 min)

Consider the ODE $u''(x) = -1$ on $[0, 1]$ with boundary conditions $u(0) = u(1) = 0$.

1. Write down the PINN residual $\mathcal{R}(x; \theta) = \ldots$ using a neural network $\mathcal{N}(x; \theta)$.
2. Write the total PINN loss as a sum of a PDE residual term and a boundary term.
3. What is the exact solution?

Solution: Writing a PINN Loss

Answer

1. $\mathcal{R}(x;\theta) = \frac{\partial^2 \mathcal{N}}{\partial x^2}(x;\theta) + 1.$
2. $\mathcal{L}(\theta) = \underbrace{\frac{1}{N_r}\sum_{i=1}^{N_r}\left|\mathcal{R}(x_i;\theta)\right|^2}_{\text{PDE residual}} + \lambda \underbrace{\left[\mathcal{N}(0;\theta)^2 + \mathcal{N}(1;\theta)^2\right]}_{\text{boundary conditions}}$
3. $u(x) = \frac{1}{2}x(1-x)$ satisfies $u'' = -1,\ u(0) = u(1) = 0.$ ✓

DEQN vs. PINN: Side-by-Side

DEQN (Discrete Time)

Economic eqns $G(\cdot) = 0$

$\downarrow$

$\mathcal{N}_\theta(\boldsymbol{x}_t) \approx p(\boldsymbol{x}_t)$

$\downarrow$

$\ell = \|G(\boldsymbol{x}, \mathcal{N}_\theta)\|^2$

$\downarrow$

SGD $\rightarrow \theta^\star$

same principle

$\longleftrightarrow$

+ autograd derivatives

PINN (Continuous Time)

PDE $\mathcal{D}[u] = 0$ + BCs

$\downarrow$

$\mathcal{N}_\theta(\boldsymbol{x}) \approx u(\boldsymbol{x})$

$\downarrow$

$\ell = |\mathcal{D}[\mathcal{N}_\theta]|^2 + \lambda|\mathrm{BC}|^2$

$\downarrow$

SGD $\rightarrow \theta^\star$

Both approaches are unsupervised: the physics (economics) *is* the loss function. The key addition for PINNs: we differentiate the network via `torch.autograd.grad`.

Part II

PINN Fundamentals

Automatic Differentiation for PDEs

Key tool: `torch.autograd.grad` with `create_graph=True`.

```
# First derivative dy/dx
dydx = torch.autograd.grad(
    y, x,
    grad_outputs=torch.ones_like(y),
    create_graph=True # keep graph for higher derivs
)[0]
```

- `create_graph=True`: retains the computational graph so we can differentiate again
- This gives us algorithmic derivatives (up to floating-point precision)—no finite differences!
- Works for any order: $\frac{\partial^n}{\partial x^n}$ by chaining n calls

Computing Second Derivatives

Chain two autograd calls to get $y''(x)$:

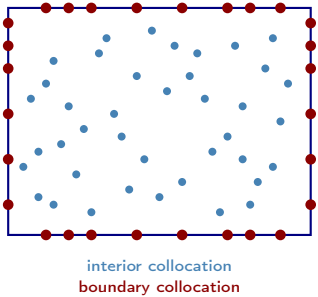
```
# First derivative
dydx = torch.autograd.grad(
    y, x, grad_outputs=torch.ones_like(y),
    create_graph=True)[0]

# Second derivative
d2ydx2 = torch.autograd.grad(
    dydx, x, grad_outputs=torch.ones_like(dydx),
    create_graph=True)[0]
```

Collocation Points

PINNs evaluate the PDE residual at randomly sampled points—no grid required.

1D: interior + boundary

2D: $\Omega = (0, 1)^2$



Points are resampled periodically during training to avoid overfitting to a fixed set.

PINN Training Algorithm

Algorithm: Physics-Informed Neural Network Training

Input: PDE $\mathcal{D}[u] = 0$, BCs $\mathcal{B}[u] = 0$, network $\mathcal{N}_\theta$, learning rate η
for epoch $= 1, \ldots, E$ **do**
  Sample N_r collocation points $\{\boldsymbol{x}_i^r\}$ in domain Ω
  Sample N_b boundary points $\{\boldsymbol{x}_j^b\}$ on $\partial\Omega$
  Forward pass: compute $\mathcal{N}_\theta(\boldsymbol{x}_i^r)$ and $\mathcal{N}_\theta(\boldsymbol{x}_j^b)$
  Autograd: compute $\mathcal{D}[\mathcal{N}_\theta](\boldsymbol{x}_i^r)$ via `torch.autograd.grad`
  $\ell_\theta = \frac{1}{N_r}\sum_i |\mathcal{D}[\mathcal{N}_\theta](\boldsymbol{x}_i^r)|^2 + \frac{\lambda}{N_b}\sum_j |\mathcal{B}[\mathcal{N}_\theta](\boldsymbol{x}_j^b)|^2$
  Update: $\theta \leftarrow \theta - \eta\,\nabla_\theta \ell_\theta$
end for
Output: Trained $\mathcal{N}_{\theta^\star}$ approximating the PDE solution

Note: Unlike DEQNs, training points are not simulated from model dynamics—they are drawn uniformly (or adaptively) from the domain.

Optimizer Pipeline: Adam $\rightarrow$ L-BFGS, in FP64

Two stages.

- Adam on resampled collocation: finds a basin.
- Deterministic L-BFGS on a *fixed* grid in FP64: closes the residual; resampling per eval breaks the line search.

Why FP64?

- tanh-MLP 2nd derivatives lose $\sim$7 digits in FP32.
- L-BFGS Wolfe search compares values at exactly that scale.
- FP32 plateaus at the noise floor; FP64 keeps shrinking.

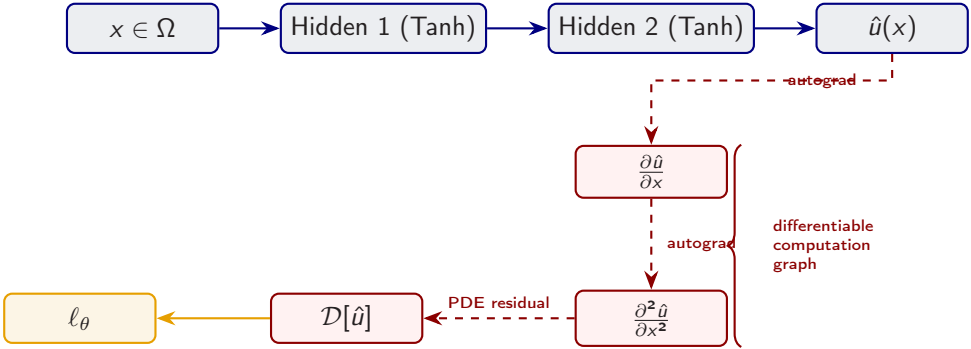
Practical recipe

1. Cast model + data to `float64`.
2. Adam, lr 10^{-3}, a few k epochs, resampled collocation.
3. Freeze grid; `torch.optim.LBFGS` (strong-Wolfe).
4. Stop on residual stagnation.

Recent precision evidence: arXiv:2505.10949.

Used in notebooks 04, 06, 07, 08: L-BFGS is the deterministic polish step. Final residuals remain problem-dependent, so report PDE residuals and solution diagnostics rather than relying on the optimizer label.

PINN Architecture

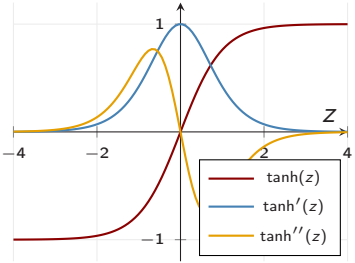


The entire pipeline—forward pass *and* derivative computation—is differentiable, enabling end-to-end gradient-based training.

Why Tanh Activations for PINNs?

PDE residuals require smooth derivatives.

- ReLU: C^0 only—second derivative is zero everywhere (useless for u'')
- Tanh: C^∞—all derivatives are well-defined and informative
- Softplus: C^∞, useful for positivity constraints (e.g. consumption)



Rule of thumb

For PINNs requiring k-th order derivatives, use activations that are at least C^k.

Warm-up: Solving an ODE with a PINN

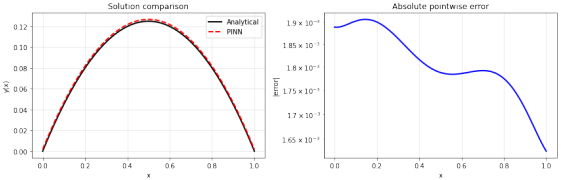
Problem: $y''(x) = -1$ on $(0,1)$, $\quad y(0) = y(1) = 0$. **Analytical:** $y(x) = \frac{1}{2}x(1-x)$.

PINN setup:

1. NN $\mathcal{N}_\theta : [0,1] \to \mathbb{R}$, 2 layers, 20 units
2. Residual: $r(x) = \mathcal{N}_\theta''(x) + 1$
3. Loss: PDE residual + BC penalty

$$\ell = \frac{1}{N}\sum_i r(x_i)^2 + \mathcal{N}_\theta(0)^2 + \mathcal{N}_\theta(1)^2$$

Notebook: `01_ODE_PINN_ZeroBCs`



Trained PINN matches analytical solution; pointwise error $\sim 10^{-3}$.

Non-Zero Boundary Conditions: The Challenge

Problem: $y''(x) = -1$ on $(0, 1)$ with $y(0) = 1$, $y(1) = 2$.

Analytical solution: $y(x) = -\frac{1}{2}x^2 + \frac{3}{2}x + 1$.

Soft BC enforcement:

$$\ell = \frac{1}{N}\sum_i r(x_i)^2 + \lambda_0\big(\mathcal{N}_\theta(0) - 1\big)^2 + \lambda_1\big(\mathcal{N}_\theta(1) - 2\big)^2$$

Issues with soft enforcement:

- BCs are only approximately satisfied
- Penalty weight λ requires tuning: too small $\Rightarrow$ BC violated; too large $\Rightarrow$ PDE ignored
- Boundary errors can propagate into the interior

$\Rightarrow$ **Motivates hard boundary enforcement** (Part III).